LECTURER: JOHN DOE

REINFORCEMENT LEARNING

REINFORCEMENT LEARNING
TOPIC OUTLINE

Introduction to Reinforcement Learning	1
Dynamic Programming and Markov Decision Processes	2
Bandit Problems	3
Q-Learning	4
Reinforcement Learning Approaches	5
Summary: Reinforcement Learning	6

UNIT 4

Q - LEARNING

STUDY GOALS



- Understand and define the **Q-function**'s role
- Explain how **neural networks** can be used in reinforcement learning
- Comprehend how **policy gradient** methods work
- Learn how to combine **value-based** learning and **policy-based** learning in actor-critic approaches

EXPLAIN SIMPLY



1. How does the Q-function guide decision-making in reinforcement learning tasks

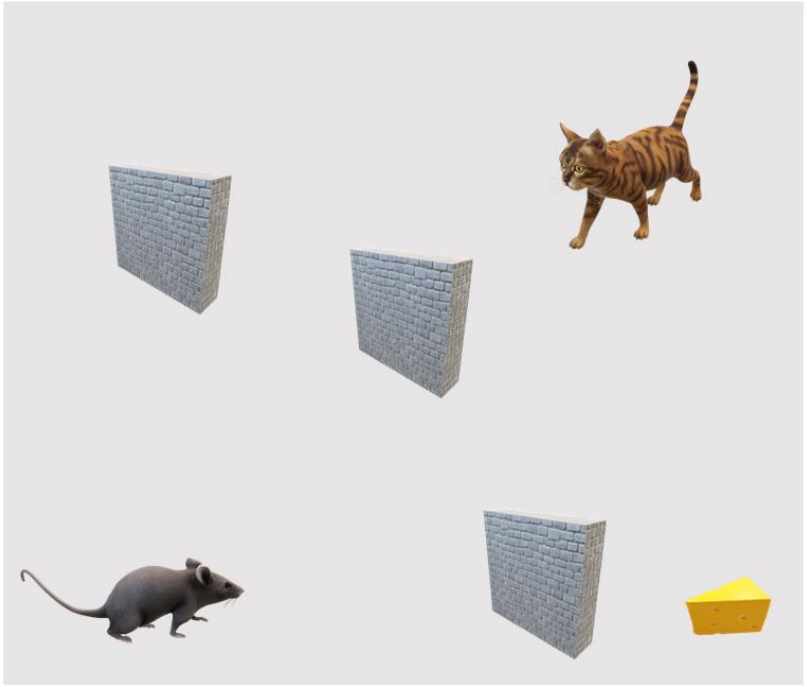
2. In what ways do neural networks enhance learning and decision-making in RL

3. What techniques leverage neural networks to enhance Q-learning's effectiveness and convergence

Game context and rules

- Cat and mouse move with 8 degrees of freedom
- Mouse avoids cat and reaches cheese
- Cat catches mouse, score changes, new round starts

Cat and Mouse Game

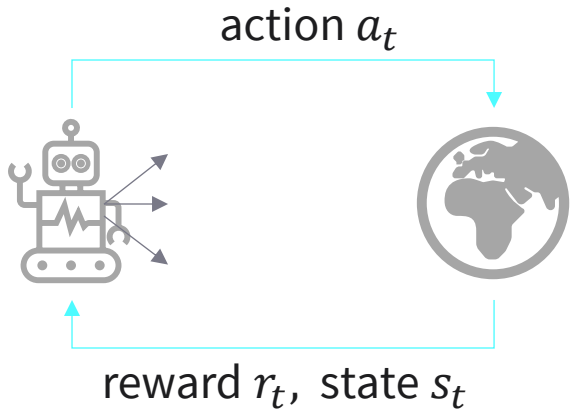


Considering uncertain cat movements, how does mouse reach the cheese ?

Source of the image: Own representation.

Optimizing computer decisions through trial-and-error learning

- **State:** current condition
- **Action:** allowed moves in each state
- **Reward:** feedback and incentive for actions
- **Policy:** decision – making strategy



- **Agent**: the learning entity
- **Environment**: external context

Source of the text: Sutton & Barto, 2018.
Source of the image: Own representation.

- **Q-factor:** reward when taking optimal action following optimal policy

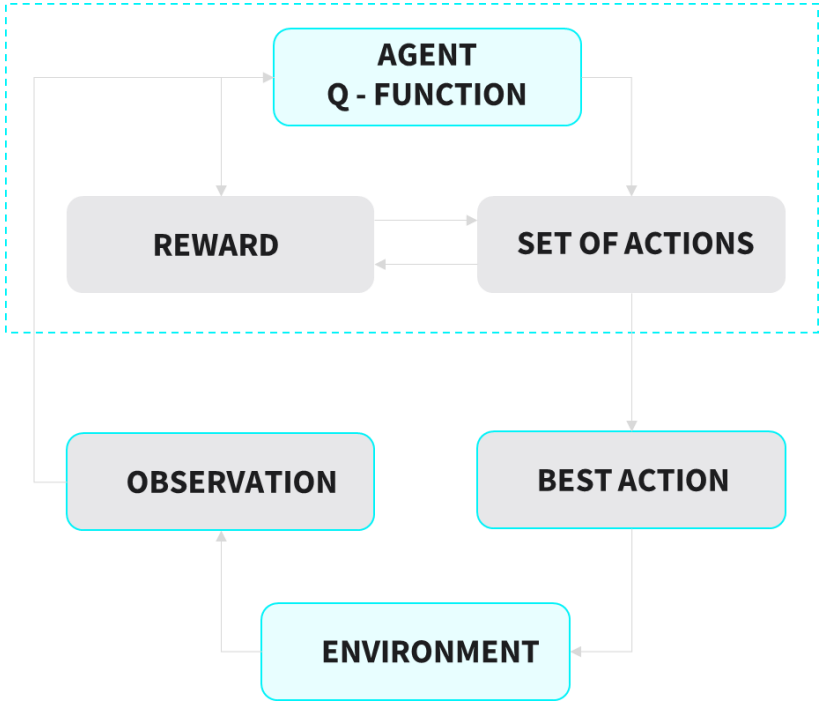
$$Q_1(s,a) \equiv r\,(s,a)$$

$$\pi_1(s) = \underset{\alpha}{\operatorname{argmax}}\, Q_1(s,a)$$

- Convergence assures optimal value function (Bellman equation)

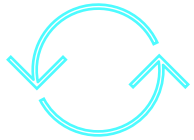
$$V^*(s) = \max_{\alpha}\left[r(s,a) + \gamma \sum_{s'} P(s'\,|s,a)\, V^*(s')\right]$$

Goal: discover optimal action strategy through iterative value updates



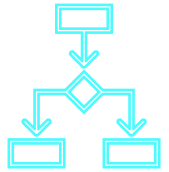
Source of the text: Sutton & Barto, 2018.
Source of the image: Own representation.

STATE VALUE ITERATION AND ACTION VALUE ITERATION



State Value Iteration

- Initialize state values
- Iteratively update state values: incorporate max expected rewards and future state values
- Stop when convergence

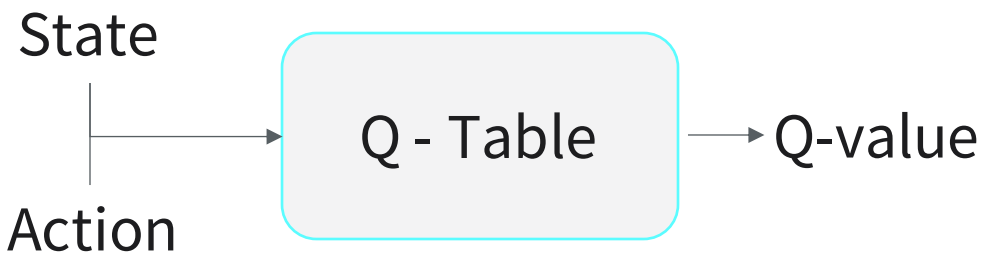


Action Value Iteration

- Initialize Q-values
- Iteratively update Q-values: incorporate immediate reward and future Q-values
- Stop when convergence

Q-LEARNING ALGORITHM

Learn optimal actions through iterative Q-value updates and exploration



1. Initialize $Q(s,\ a)$ arbitrarily
2. Start at initial state s
3. Repeat until terminal state reached
   - Choose action α for state s
   - Take action α, observe reward r, and move to new state s'
   - Update Q-values

$$Q(s,a) = Q(s,a) +$$

$$\alpha\left[r + \gamma\ \max_{\alpha} Q(s',a) - Q(s,a)\right]$$

4. Set the current state to new state $s \mapsto s'$

Source of the text: Sutton & Barto, 2018

SARSA ALGORITHM

- **Q-learning (off policy)**
  values updated independently of current policy

- **SARSA (on policy)**
  next action follows same policy

1. Initialize $Q(s,\ a)$ arbitrarily
2. Start at initial state s
3. Repeat until terminal state reached
   - Choose action α for state s
   - Take action α, observe reward r, and move to new state s'
   - Choose action a' for new state s'
   - Update Q-values

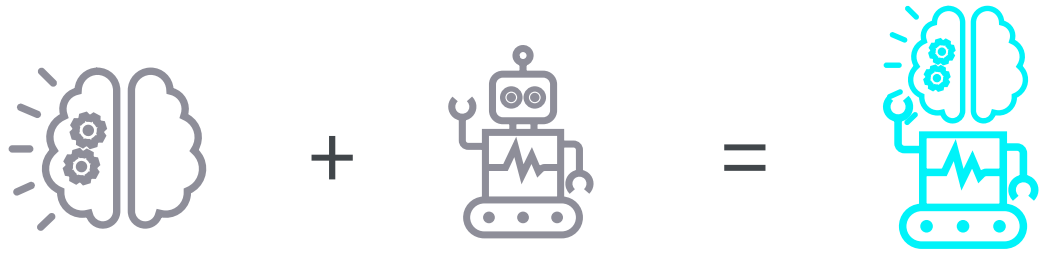
$$Q(s,a) = Q(s,a) +$$

$$\alpha\left[r + \gamma\ \max_{\alpha} Q(s',a') - Q(s,a)\right]$$

4. Set current state to new state $s \mapsto s'$ and current action to new action $\alpha \mapsto \alpha'$

Source of the text: Sutton & Barto, 2018

Synergy of neural networks and reinforcement learning



Neural Network

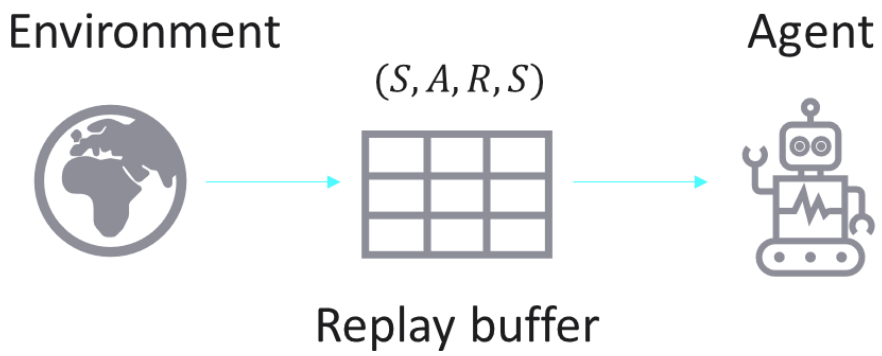
Reinforcement Learning

Deep Reinforcement Learning

- Impractical to build Q-table explicitly
- State-action space grows too big
- Approximate Q-function using neural networks
- Learn Q-values from state – action pairs

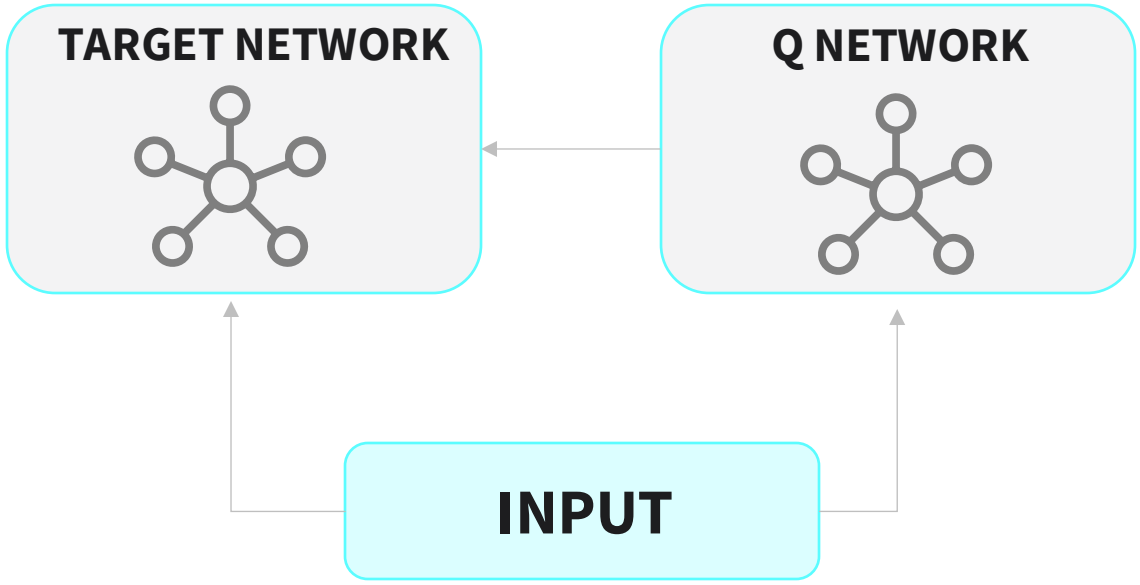
Experience Replay

- Save transitions $(S_t, A_t, R_{t+1}, S_{t+1})$ into buffer and sample batch
- Use batch to train agent



Double Q-Learning

- Utilize separate target and value networks

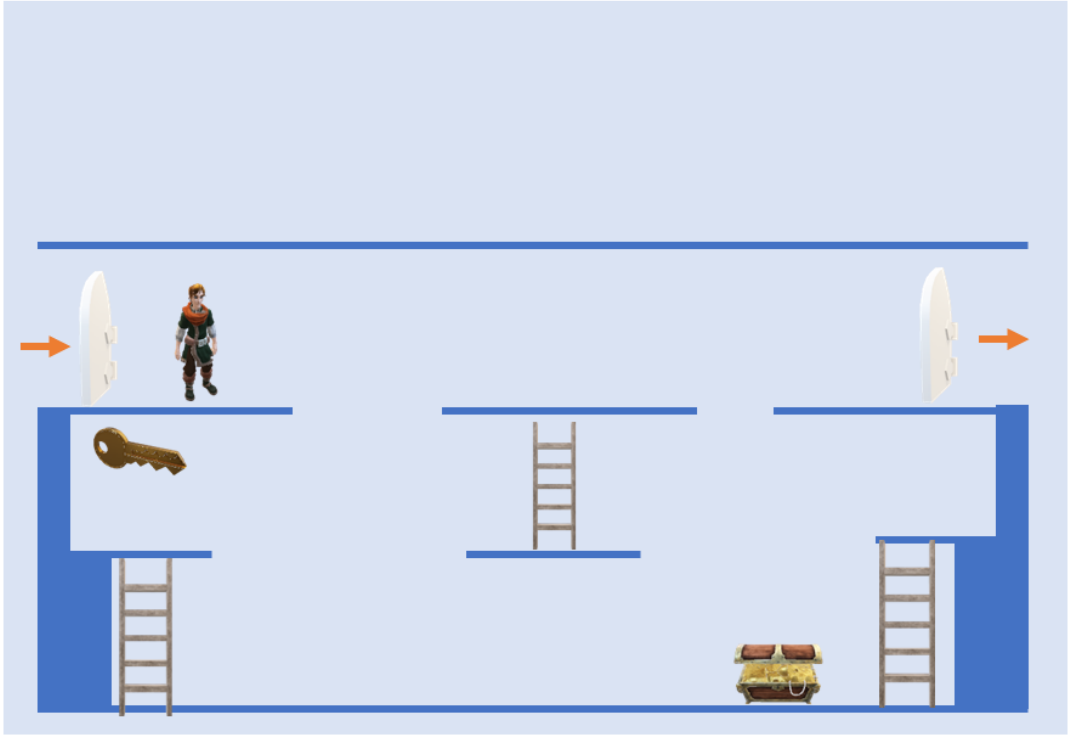


Delayed sparse rewards

- Hard learning when rewards delayed or sparse
- Exploration challenging, requiring new strategies

Hierarchical learning

- Complex environments: multiple objectives, interdependencies
- Hierarchy: meta-controller sets goals, sub-controller achieves them

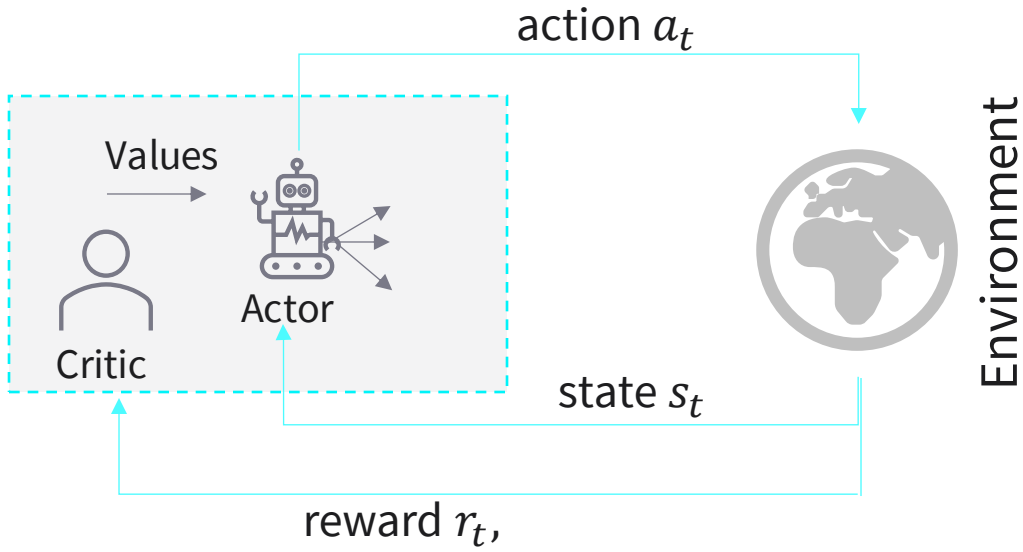


Layout of a complex games with multiple sub-goals

Source of the image: Own representation.

Simultaneous policy evaluation and improvement
Combine policy-based and value-based learning

- **Actor** learn policy (**action selection**)
- **Critic**: estimate value function (**policy evaluation**)
- Enhances stability, convergence, and exploration



Source of the image: Own representation.

REVIEW STUDY GOALS



- Understand and define the **Q-function**'s role
- Explain how **neural networks** can be used in reinforcement learning
- Comprehend how **policy gradient** methods work
- Learn how to combine **value-based** learning and **policy-based** learning in actor-critic approaches

SESSION 4

TRANSFER TASK





Case study

Your task is to develop an AI-agent that can play the Atari game Pong using deep Q-learning. Your goal is to develop an optimal policy that maximizes the game score

Task

What key steps will you consider when implementing Q-learning? Which neural network architecture would be most appropriate? How to optimize Q-learning? Consider factors such as experience replay and target network updates, among others

TRANSFER TASK
PRESENTATION OF THE RESULTS

Please present your results.

The results will be discussed in plenary.





1. Suppose we are using policy iteration to find the optimal policy of MDP. This results in a sequence of policy functions $\varphi[1], \varphi[2], \ldots, \varphi[n]$. After the nth iteration we notice that $\varphi[n-1] \neq \varphi[n]$, but that the policy value functions of these two policies are equal. What should we conclude?
   a) An optimal policy doesn‘t exist
   b) We have not converged to an optimal policy
   c) $\varphi[n]$ is an optimal policy but $\varphi[n-1]$ is not
   d) Both $\varphi[n-1]$ and $\varphi[n]$ are optimal policies



2. A minimum discounted cost MDP has a state space $\chi = \{1,2,3,\ldots,100\}$. From state $x = 2$ two actions are available. If action $a = 1$ is chosen, the process transitions to states $x = 1$ or $x = 3$ with probability 1/2 each, and a stage cost $R(2,1) = 5$ is accrued. If action $a = 2$ is chosen, the process transitions to state $x = 1$ with probability once, and a stage cost of $R(2,2) = 10$ is accrued. The discount factor is $\beta = 0.9$. Then suppose policy iteration is used to evaluate the optimal policy. The policy value function for the nth policy iterate $\varphi[n]$ has a policy value function with values $V'(1) = 20.5$, $V'(3) = 36.4$. What action will the next iterate $\varphi[n+1]$ choose from state $x = 2$? Justify your answer

   a) Action a = 1 only
   b) Action a = 2 only
   c) Both actions are equally optimal
   d) There is not enough information to determine the optimal action



3. A multilayer perceptron has an input layer with 5 nodes, a hidden layer with 16 nodes, and an output layer with 2 nodes. What is the total number of weight and bias parameters
   a) 23
   b) 41
   c) 130
   d) 178

LIST OF SOURCES

Sutton, R. S., & Barto, A. G. (2018a). Reinforcement learning: An introduction (2nd ed.). MIT press.

© 2022 IU Internationale Hochschule GmbH
This content is protected by copyright. All rights reserved.
This content may not be reproduced and/or electronically edited, duplicated, or distributed in any kind of form without written permission by the IU Internationale Hochschule GmbH.